

Instituto Tecnológico de Costa Rica

Fundamentos de sistemas computacionales

Profesor: Luis Alonso Barboza Artavia

Grupo 3

Proyecto #2 CElect

Estudiantes:

Leonardo Borbón Mena

Carné: 2026105254

Santiago Allón Quesada

Carné: 2026099579

Introducción:

El siguiente documento se refiere a la documentación del Proyecto 2 de Fundamentos de Sistemas Computacionales, y los resultados obtenidos en este. El mismo abarca la solución propuesta por el grupo, conclusiones y recomendaciones del ejercicio planteado.

El proyecto se basó en la creación de un circuito que simulara la función de una máquina expendedora, en la que se eligieron productos, se presentara la cantidad de estos, se pudiera comprar y entregar el producto, todo esto monitoreado por un computador, el cual puede detener el funcionamiento de la máquina al ponerla en modo mantenimiento, como también observar las ventas históricas de la máquina.

El ejercicio propuesto se logró satisfactoriamente, con un apoyo continuo entre el grupo para lograr un trabajo completo.

Solución del proyecto:

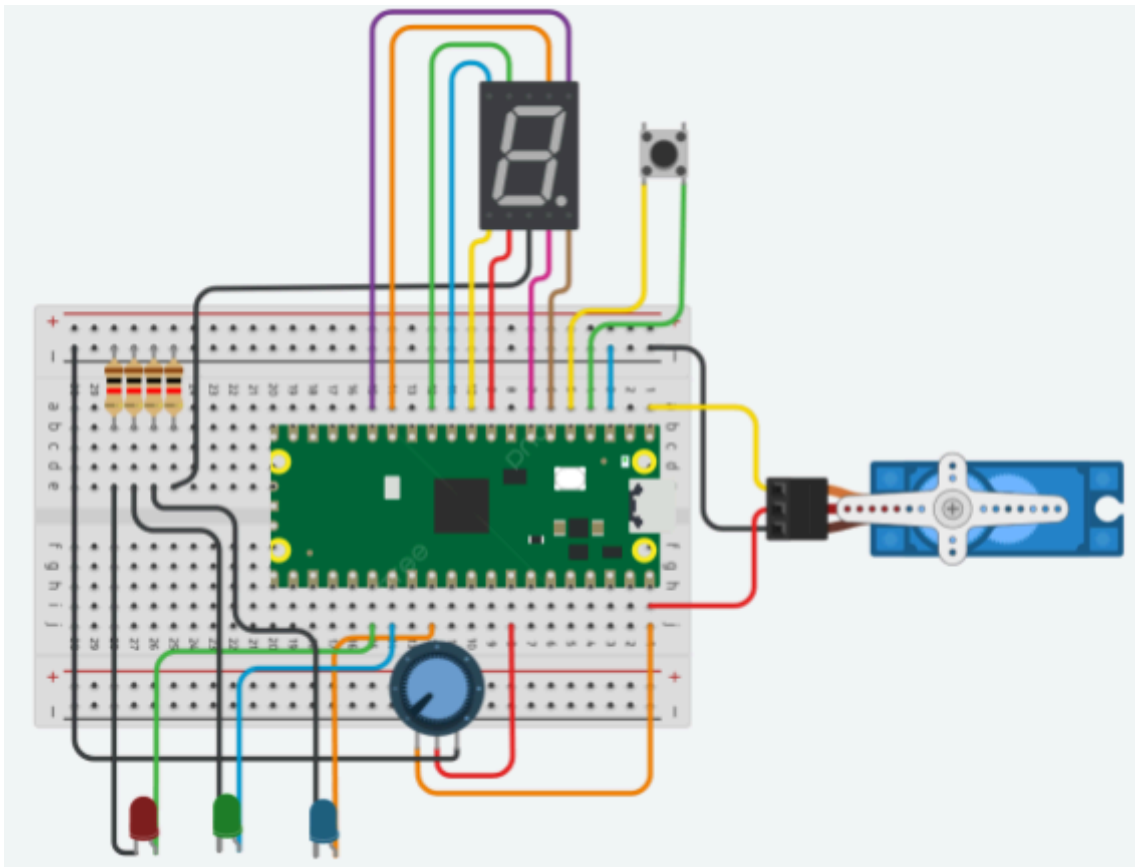
Para la programación del proyecto se usó el lenguaje Python, como también Micro Python para la utilización del microcontrolador, en estos se escribió todo el programa. Además de la utilización de librerías, tales como tkinter, time, threading, requests, socket y network. Por otro lado, se utilizaron archivos json para guardar los registros de ventas necesarios. El microcontrolador utilizado fue una Raspberry Pi Pico W.

Para la realización del proyecto se utilizaron diversos componentes, tales como:

1. Potenciómetro: Este permite, mediante la medición del dato que proporciona, determinar el producto seleccionado por el usuario.
2. Servomotor: Se usa para abrir la compuerta en donde salen los productos al ser comprados.
3. LED: Hay 3 presentes, rojo, verde y azul, tienen la función de decir si hay stock, si no hay, o si la máquina está en modo mantenimiento, respectivamente.

4. 7 Segment-Display: Permite comunicar al usuario la cantidad de existencias del producto que eligió.
5. Botón: Permite al usuario comprar el producto seleccionado.
6. Raspberry Pi Pico W: El microcontrolador utilizado, se usó como forma de comunicación entre PC y máquina, y como unidad de procesamiento.

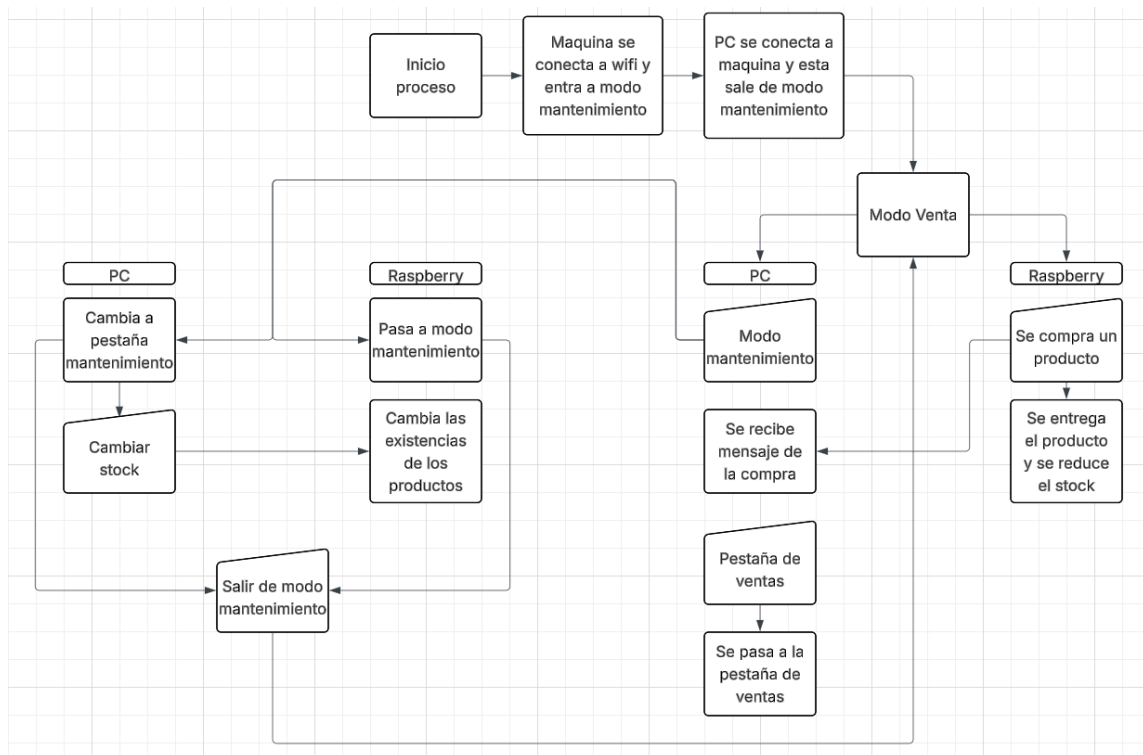
Imagen 1. Diagrama del circuito utilizado



En la **Imagen 1** se observa el diagrama del circuito utilizado para el proyecto, en el que se observan las conexiones entre potenciómetro, servomotor, LED 's, botón y 7 Segment-Display. Se observa que la Raspberry controla cada uno de los componentes para que funcionen cuando deben.

El diagrama del circuito funcionó como base para construir él mismo en una protoboard, antes de proseguir con la construcción de una caja en la que se pudieran poner los componentes. Para finalmente instalar estos en la maqueta construida.

Imagen 2. Diagrama de arquitectura



En la **Imagen 2** se observan los modos que posee tanto el PC como la Raspberry, en donde se presenta un ciclo que permite la función constante de la máquina expendedora. Se observa que al inicializar la máquina expendedora, esta se queda en modo mantenimiento hasta que se establezca una conexión con el PC, además, al llegar al modo de venta, se pueden realizar varias acciones:

1. Cuando un usuario compra un producto, el stock se reduce y se envía un mensaje al PC notificando de la venta
2. Se puede cambiar a una pestaña de ventas en donde se puede ver el registro histórico de ventas de la máquina expendedora, como también el dinero producido, tanto en colones como en dólares.
3. Se puede pasar a modo mantenimiento, en donde la PC pasa a una pestaña específica de mantenimiento y la máquina expendedora detiene sus operaciones hasta que sea notificada que se volvió al modo de venta.

En el modo mantenimiento, se pueden realizar otras acciones:

1. El modo mantenimiento permite que desde la PC se introduzcan nuevos valores de stock, en este caso, se utiliza como un modo para que una persona que repone la máquina pueda cambiar los valores de stock a los valores actuales reales.
2. Salir de mantenimiento envía un mensaje a la máquina expendedora para que esta vuelva a su estado de venta y continúe con sus operaciones normales.

Algunas de las principales funciones en el programa son:

Principales Funciones:

Raspberry Pi Pico W

```
def mostrar_cantidad_productos(producto):  
    global lista_productos  
    numero = numeros_segmento[str(lista_productos[producto])]   
    for x in range(7):  
        diccionario_segmentos["segmento" + str(x)].value(numero[x])  
#Strx es por cuestion de ir recorriendo cada segmento y cambiarle  
so valor
```

Esta función permite mostrar las existencias del producto en el 7 Segment-Display. Funciona con dos diccionarios que permiten determinar la serie que se tiene que realizar en el display para que se muestre el número específico.

```
def iniciar_servidor_async(ip):  
    #Esta funcion la tome del proyecto de StrangerTEC  
  
    s = socket.socket() #Crea un socket, en este caso para la  
conexion asincrona
```

```

        s.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
#Permite reusar el puerto si la raspberry se desconecta

        s.bind((ip, PORT)) #Le asocia la ip y el puerto al socket creado

        s.listen(1) #Significa que escuchará solo un mensaje a la vez

        poller = select.poll() #Crea un poller

        poller.register(s, select.POLLIN) #POLLIN revisa si hay un mensaje, pero lo hace una vez y no en un loop constante, dando la logica de la conexion asincrona

        print("Servidor Asincrono Listo")

        return s, poller #Devuelve ambos para su uso posterior

```

Esta función permite crear un servidor asíncrono en la raspberry, permitiendo que esta misma espere mensajes y opere el hardware en un mismo while loop.

```

def recibir_mensaje(data):

    global lista_productos, modo_ventas, modo_mantenimiento, led_azul

    mensaje = data.decode().strip().upper()

    if mensaje.startswith("STOCK:"):

        mensaje_lista_stock = mensaje.split(":") #Trabnsformarlo en una lista

        lista_productos = [] #Se va actualizar la lista de productos, entonces se limpia

        for i in range(1, len(mensaje_lista_stock)):

            lista_productos.append(int(mensaje_lista_stock[i]))

        print(lista_productos)

```

```

        if modo_mantenimiento == False: #esto es para prevenir que se quite el mantenimiento durante la ejecucion

            modo_ventas = True

            led_azul.value(0)

        elif mensaje.startswith("MANTENIMIENTO:ACTIVAR"):

            iniciar_mantenimiento()

        elif mensaje.startswith("MANTENIMIENTO:DESACTIVAR"):

            led_azul.value(0)

            servo.duty_u16(half_duty)

            modo_mantenimiento = False #Le deja saber al sistema si esta en mantenimiento o no

            modo_ventas = True

```

La función de recibir mensajes cobra sentido después de la función del server asíncrono, ya que esta misma se encarga de interpretar los mensajes recibidos del PC para actualizar stock o cambiar entre el modo de ventas o el modo de mantenimiento

```

while True:

    # Eventos de red

    eventos = poller.poll(0) #Pregunta si hay actividad sin bloquear el sistema

    for sock, ev in eventos: #Itera sobre cada socket para recibir su informacion

        #Conexion al servidor

        if sock == server_sock: #Basicamente dice que si el sock que tuvo actividad es el mismo que el de la Rasp, es porque hay un cliente conectandose

```

```

        cliente_sock, addr = server_sock.accept() #Acepta la
conexion

        print("PC Conectada:", addr)

        poller.register(cliente_sock, select.POLLIN) #Registra
un poller, que sera la entrada de datos desde la PC

    elif sock == cliente_sock: #Si llega un mensaje desde el PC

        try:

            data = cliente_sock.recv(1024) #Intenta recibir
informacion

            if data:

                #Si hay informacion, simplemente llamamos a la
funcion de manejar_mensajes

                recibir_mensaje(data)

        except: #Si no hay sock del cliente, simplemente lo
desconecta

            poller.unregister(cliente_sock) #quita el poller

            cliente_sock.close() #cerramos el socket del
cliente

            cliente_sock = None #lo pasamos a None,

# Modo de ventas

if modo_ventas:

    # Potenciometro

    adc_value = potenciometro.read_u16()

    if adc_value > 50000:

        producto = 0 #Este es el indice de la lista

    elif 20000 < adc_value < 50000:

```



```

        producto = 1

    else:

        producto = 2

# Prender Leds

    if lista_productos[producto] == 0: #Si no hay producto,
prender rojo y no prender verde

        led_rojo.value(1)

        led_verde.value(0)

    else: #Si hay productos, hacer la instruccion alreves

        led_rojo.value(0)

        led_verde.value(1)

    mostrar_cantidad_productos(producto) #Mostrar la cantidad
de producto en el 7 segmentos

# Procedimiento compra productos

if boton.value() == 1 and lista_productos[producto] > 0:

    lista_productos[producto] = lista_productos[producto] -
1

    enviar_mensaje("VENTA", producto)

    if lista_productos[producto] == 0: #Si no hay producto,
prender rojo y no prender verde

        led_rojo.value(1)

        led_verde.value(0)

    else: #Si hay productos, hacer la instruccion alreves

        led_rojo.value(0)

        led_verde.value(1)

```

```

        mostrar_cantidad_productos(producto) #Mostrar la
cantidad de producto en el 7 segmentos

        servo.duty_u16(min_duty)

        time.sleep(3)

        servo.duty_u16(half_duty)

        time.sleep(1)

    time.sleep(0.1) #Evitar que la raspberry se sobre caliente

```

Por último, el while loop es el que permite una continua ejecución del código, primero revisando si hay un mensaje y luego ejecutando las tareas relacionadas a la venta de la máquina

Programa PC

```

def obtener_tipo_cambio():

    global tipo_cambio

    #Por si falla el API para volver a intentarlo (en sentido de
ejecucion DESPUES de la ejecucion inicial al arrancar el programa)

    tipo_cambio = "Obteniendo tipo de cambio..."

    if ventana_actual == "STATS": #Si no esta en STATS, entonces no
es necesario refrescar la pagina

        modo_estadisticas() #Refrescar para eliminar el boton

    #Procedimiento

    try:

```

```

        # Realiza una solicitud HTTP a la API y espera 5 segundos
        (HTTP --> mensaje que un cliente envia a un servidor para pedir un
        recurso o realizar una accion)

        response =
requests.get("https://exchange.r.albertosaenz.com/api/tipodecambio"
,timeout=5) #Espera 5 segundos

        if response.status_code == 200: #Pregunta si el codigo de
status del response es 200, si no significaria que no se acceso
correctamente a la API

            datos = response.json() #Datos es un diccionario y le
aplicamos .json() para decirle a Python como es el formato de los
datos recibidos

            for divisa in datos: #Como recibimos una lista de
diccionarios, simplemente lo iteramos

                if divisa["divisa"] == "USDollar": #El API usado
entrega dolares, entonces esto de aquí es preguntarle al
diccionario "Que informacion contiene "divisa"

                    tipo_cambio = float(divisa["venta"]) #Si es el
USDollar, simplemente se saca un valor en float y listo.

                    if ventana_actual == "STATS": #Refrescar la
pagina

                        modo_estadisticas()

                        break

            else : #Es decir que no se acceso correctamente la pagina

                tipo_cambio = "ERROR" #es decir que fallo haciendo
esto, sirve para poner el boton para reintentar

                if ventana_actual == "STATS":

                    modo_estadisticas()

except Exception as e: # En caso de que falle la ejecucion

    print("Error obteniendo tipo de cambio:", e)

```

```

        tipo_cambio = "ERROR" #es decir que fallo haciendo esto, sirve para poner el boton para reintentar

        if ventana_actual == "STATS":

            modo_estadisticas()

        return None

```

Esta función le permite al programa conectarse a una API para obtener el tipo de cambio de dólares a colones actualizado a la fecha, el cual se usa para calcular las ganancias del usuario en dólares.

```

# Enviar productos

def enviar_mensaje_rasp(id): #Funciona con id, es como decir "se presiono el boton 1, haga 1, ... se presiono boton n, haga n"

    # DEFINIR MENSAJE

    if id == 1:

        mensaje = "STOCK" # Empezar con el mensaje

        for producto in lista_productos: #Para cada producto se usa algo

            mensaje += f"{producto}"

    elif id == 2:

        mensaje = "MANTENIMIENTO:ACTIVAR"

    elif id == 3:

        mensaje = "MANTENIMIENTO:DESACTIVAR"

    # INTENTAR MANDAR MENSAJE

    try:

        client_socket.send(mensaje.encode())

    except:

        print("Error enviando mensaje")

```

enviar_mensaje_rasp(id) es una función principal ya que permite que la PC actualice el STOCK de la maqueta y también permite mandar el mensaje de activar o desactivar el mantenimiento, permitiendo una comunicación bidireccional entre ambos dispositivos\

```
def inicio():  
    leer_archivos()  
    threading.Thread(target=connect, daemon=True).start()  
    threading.Thread(target=obtener_tipo_cambio,  
daemon=True).start()  
    menu_principal()
```

La función inicio() permite hacer las llamadas necesarias para iniciar el programa de manera correcta, usando múltiples hilos de procesamiento para evitar congelar la funcionalidad de la interfaz gráfica.

Conclusiones:

Se concluyó con el proyecto de forma satisfactoria, al construir un circuito capaz de lo requerido, apoyado en diagramas de arquitectura y diagramas de circuitos.

Se logró utilizar componentes analógicos, tales como el potenciómetro y servomotor, mediante el estudio y análisis de sus funcionalidades, permitiendo así utilizarlos de manera eficaz, tanto para este proyecto como para otros en el futuro.

Se observó la importancia en la creación de diagramas de arquitectura y diagramas de circuitos para una previsión de lo que se realizará en el proyecto, logrando una visión más centrada del mismo, como también más seguridad en lo que se está realizando.

Recomendaciones:

Se recomienda utilizar materiales más fuertes para la creación de la maqueta, para que esta resista más y sea más fácil de usar, estos podrían ser cartón reciclado, o alguna forma de impresión en 3D, además, debería decorarse.

Para que los componentes sean más fiables, deberían soldarse para que estos no puedan soltarse, ya que, al usar jumpers, existe la posibilidad de que estos se suelten y por ende hagan que se dañen las funcionalidades de la máquina.

Bibliografía:

Declaración de uso de IA

Durante el desarrollo del proyecto, se utilizaron distintas herramientas de inteligencia artificial como apoyo para el debugging, asistencia lógica en la programación y comprensión del funcionamiento de una API. Todas las implementaciones finales, integración de recursos y modificaciones al proyecto fueron realizadas manualmente por los estudiantes.

Herramienta IA	Uso realizado	Resultado obtenido	Intervención del estudiante
ChatGPT	Explicación de APIs REST, Tkinter, sockets, JSON y debugging	Comprensión de conceptos, ejemplos	El código final fue implementado, de adaptado y probado manualmente

código y apoyo
técnico

Claude Asistencia en Sugerencias de Las decisiones de
lógica de estructura y diseño e
programación y corrección de implementación
revisión de errores fueron realizadas por
soluciones el estudiante

APIs y servicios externos utilizados

API de Tipo de Cambio

- Fuente: <https://exchange.cr.albertosaenz.com/api/tipodecambio>
- Uso: Obtención del tipo de cambio CRC/USD en tiempo real.
- Método HTTP: GET.
- Formato de respuesta: JSON.
- Aplicación dentro del proyecto: Conversión de ganancias de colones a dólares.
- Link del Repositorio de Github :
<https://github.com/DaveSV/Indicadores-Econ-micos-BCCR-Api/tree/main>

Referencias

Anthropic. (2026). Claude (claude-sonnet-4-6) [Modelo de lenguaje de IA].
<https://claude.ai>

OpenAI. (2026). ChatGPT (GPT-4) [Modelo de lenguaje de IA].
<https://chat.openai.com>

Raspberry Pi Pico and Potentiometers. (s. f.). Halvorsen.blog. Recuperado 6 de
junio de 2026, de
https://www.halvorsen.blog/documents/technology/iot/pico/pico_potential_meter.php

Sáenz, A. (s.f.). *Indicadores-Econ-micos-BCCR-Api* [Código fuente]. GitHub.
<https://github.com/DaveSV/Indicadores-Econ-micos-BCCR-Api/tree/main>

Santos, S. (2024, mayo 23). *Raspberry Pi Pico: Servo motor (Micro Python)*.
Random Nerd Tutorials.
<https://randomnerdtutorials.com/raspberry-pi-pico-servo-motor-micropython/>